



Universidad
Nacional
de Rosario



TECNICATURA UNIVERSITARIA EN INTELIGENCIA ARTIFICIAL (TUIA)

PROCESAMIENTO DE IMÁGENES(PDI)

Trabajo práctico Nro . 2

Grupo Nro 1:

ELIZONDO RIOS, Iñaki Cruz	Legajo: E-1260/2
JUAREZ, Leandro Santiago	Legajo: J-0633/5
TORINO, Facundo	Legajo: T-3132/1

**PRELIMINAR: los archivos de resolución pueden encontrarse en :
https://github.com/Ftedp/Procesamiento_de_Imagenes_TP2**

PRESENTACIÓN DEL PROBLEMA 1 :

El problema presentado consiste en a partir de una imagen de una cinta transportadora transportadora industrial, segmentar el área de interés, detectar y segmentar cada uno de los objetos transportados, en esta caso pastillas y cápsulas, realizar una clasificación de pastillas por sus características morfológicas y cromáticas, y hacer un recuento de los objetos encontrados.

imagen de entrada a resolver



IMPLEMENTACIÓN DE LA SOLUCIÓN:

Detección de la zona de la cinta transportadora (ROI- General):

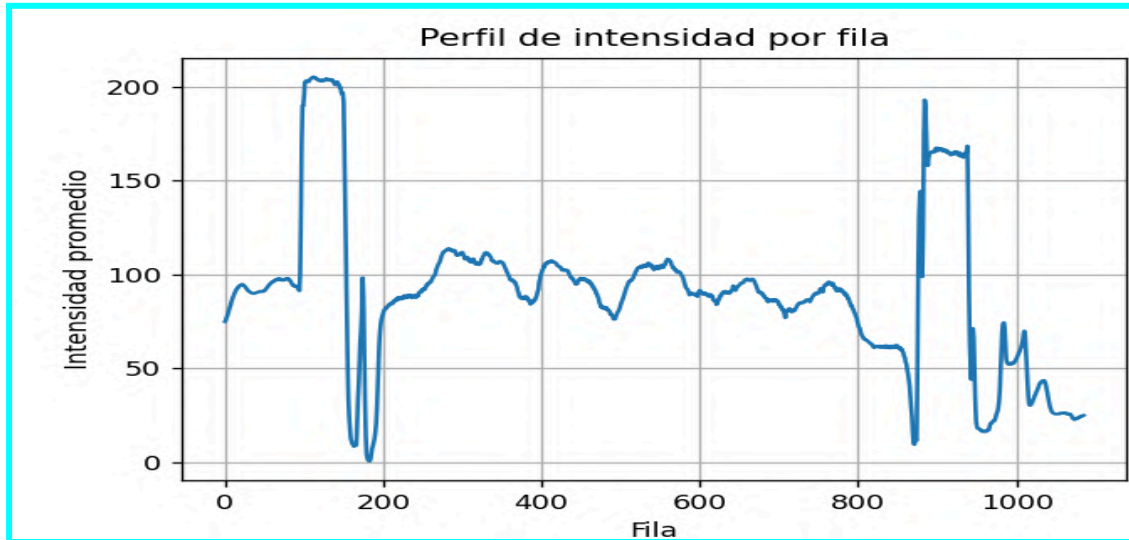
El primer desafío está en detectar la zona de interés en se encontrarán las píldoras y pastillas válidas, en nuestro caso la decisión es detectar la zona de la cinta transportadora, por lo que tomamos la decisión de que si hubiera en la imagen un objeto compatible con una pastilla o píldora por fuera de la zona de la cinta transportadora, ya será descartada del análisis posterior. Solo nos centraremos en la zona de la cinta transportadora.

Teniendo en cuenta la descripción realizada en el anterior, las bandas metálicas horizontales que enmarcan a la “zona” de las pastillas son el margen natural para delimitar nuestra zona de interés general.

para segmentar la zona de interés general trabajamos con escala de grises a los fines de disminuir la complejidad del problema



Análisis de la intensidad de las filas , las zonas de intensidad alta marcan las bandas metálicas de la imagen

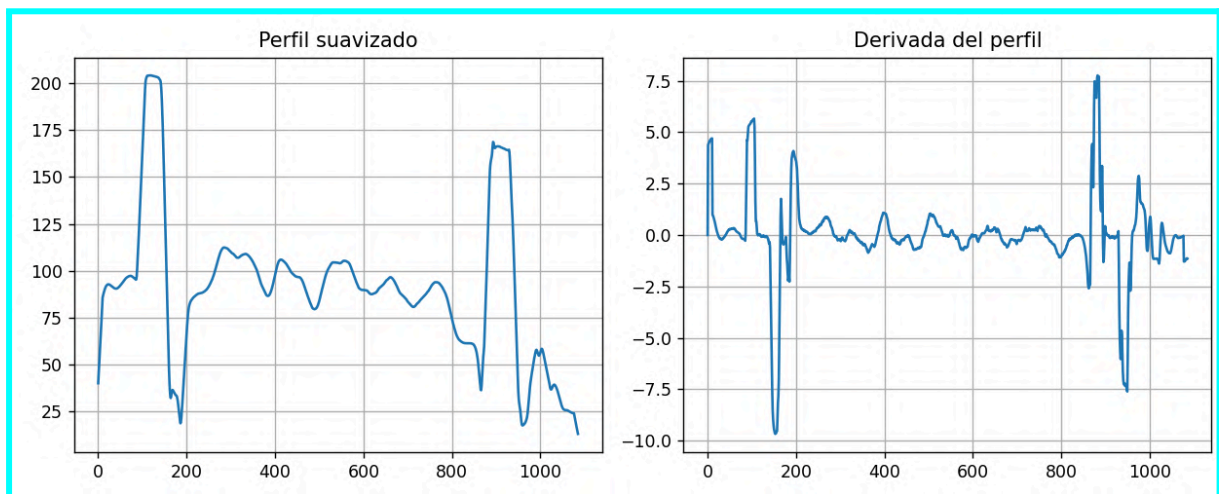


Del análisis de intensidades, vemos que las áreas que se correlacionan con la bandas metálicas se destacan por sobre el resto y delimitan el área de interés de la cinta transportadora, dejamos la advertencia que este método de delimitación es válido para una configuración similar, podría llegar a fracasas si el fondo de la cinta fuera más claro.. o las pastillas fueran más claras o más agrupadas, pero es válido para esta cinta transportadora oscura, y para esta tonalidad y concentración de pastillas.

Ajuste para delimitar la ROI -General.

En la gráfica anterior pudimos tener una aproximación a la que sería nuestra área de interés, pero como pequeñas alteraciones puntuales pueden cambiar la gráfica y nuestros “delimitadores” son una zona más o menos homogénea, vamos a realizar una acción de suavizado sobre el perfil de la imagen, para luego calcular una aproximación a la derivada del perfil de la imagen,

transformaciones sobre la señal (suavizado, análisis de la derivada)



La mayor caída nos representará un pasaje de zona clara a zona más oscura (inicio superior de la ROI - General) y la mayor subida nos representará un pasaje de zona oscura a zona clara. (límite inferior de la ROI - General)

Marcado de la zona de interés las marcas agregadas rojas marcan la zona de interés.



Área de interés segmentada



En este punto entendemos que tenemos un método robusto y automatizado para lograr la detección de la zona en donde se encuentran las pastillas y podemos concentrarnos en la detección de píldoras y pastillas.

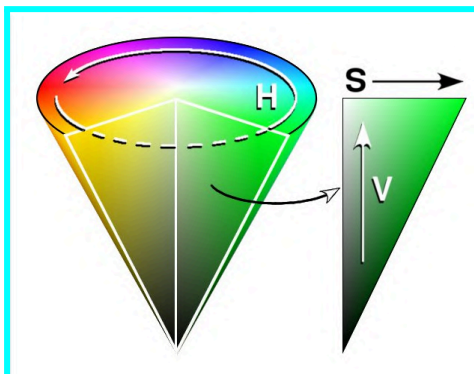
Detección de píldoras y pastillas:

Para esta parte del desafío nos centraremos en las características cromáticas y morfológicas, nos hacemos las siguientes preguntas ¿Cómo separar las pastillas del fondo ? , y ¿Cómo diferenciar los diferentes tipos de pastillas ?

Cambiando el espacio de color

Para nosotros como humanos no nos resulta difícil detectar las pastillas del fondo, pero esto es un desafío en nuestra solución del problema. Pero mirando el contexto en donde estamos trabajando vemos todas las pastillas, son más claras que el fondo, y vamos a aprovechar esa diferencia, cambiando el sistema de referencia de color de que veníamos usando hasta ahora por el espacio de color HSV .

espacio de color HSV



```
# --- Convierto ROI a HSV ---  
roi_hsv = cv2.cvtColor(roi, cv2.COLOR_RGB2HSV)  
H, S, V = cv2.split(roi_hsv)
```

Lo que hacemos con estas líneas de código es cargar nuestra roi (Roi General), la transformamos en HSV y separamos los 3 canales

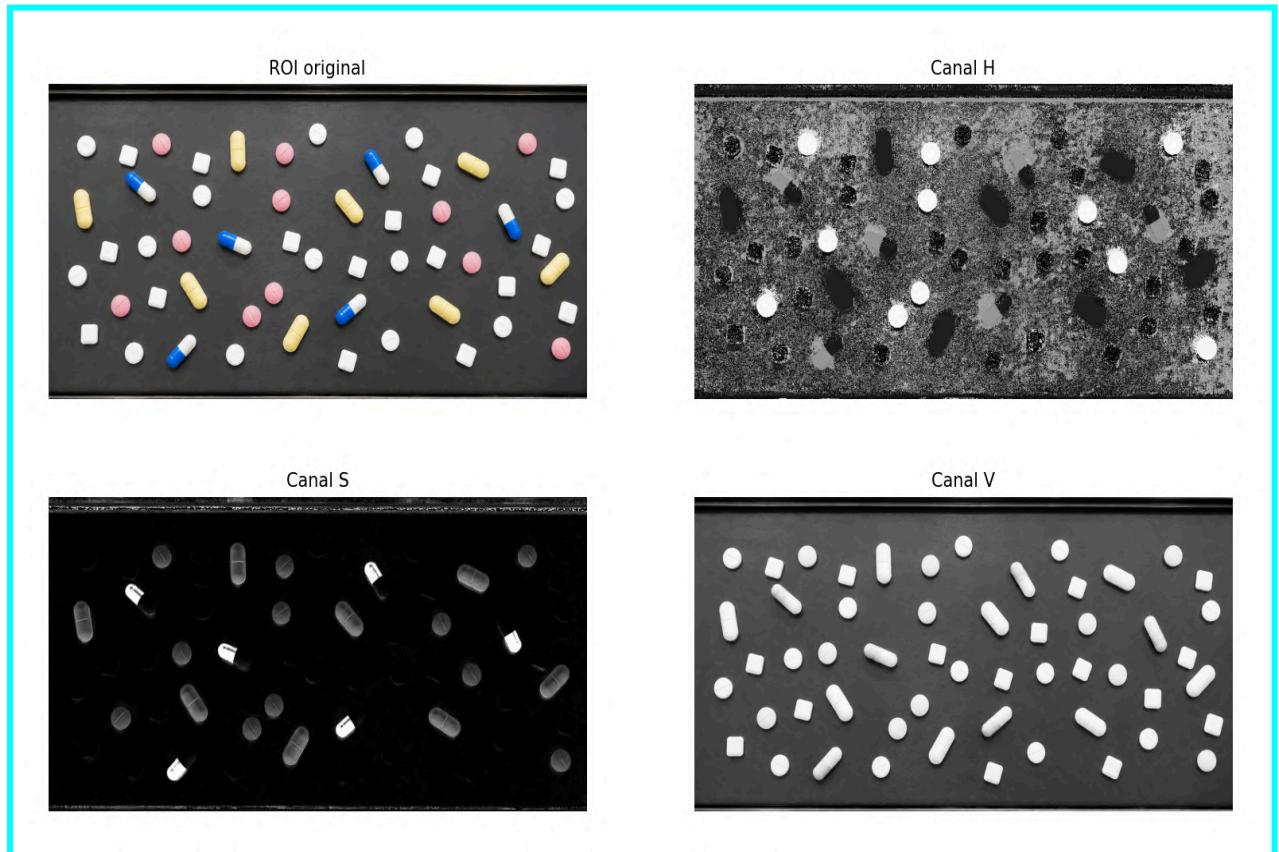
De modo simple podemos decir que cada canal responde a esta pregunta.

H, ¿sobre qué color estamos trabajando?

S, ¿qué tan saturado, “fuerte” es ese color?

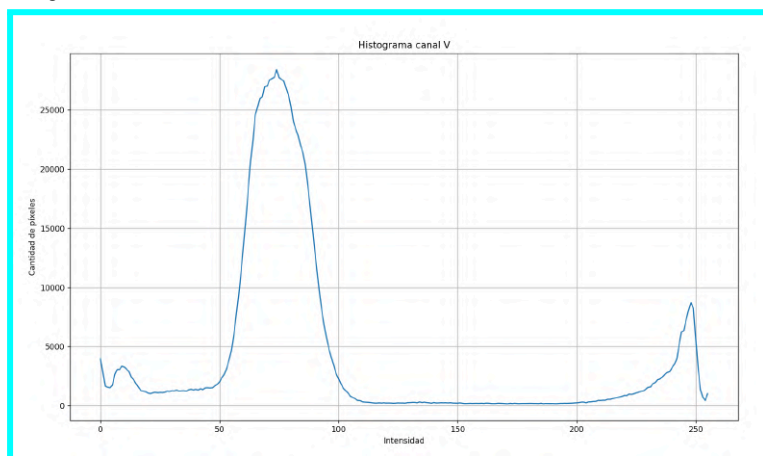
V, ¿qué tan brillante, que tanto “blanco” tiene ese color ?

Exploración de la imagen en los diversos canales del espacio de color HSV

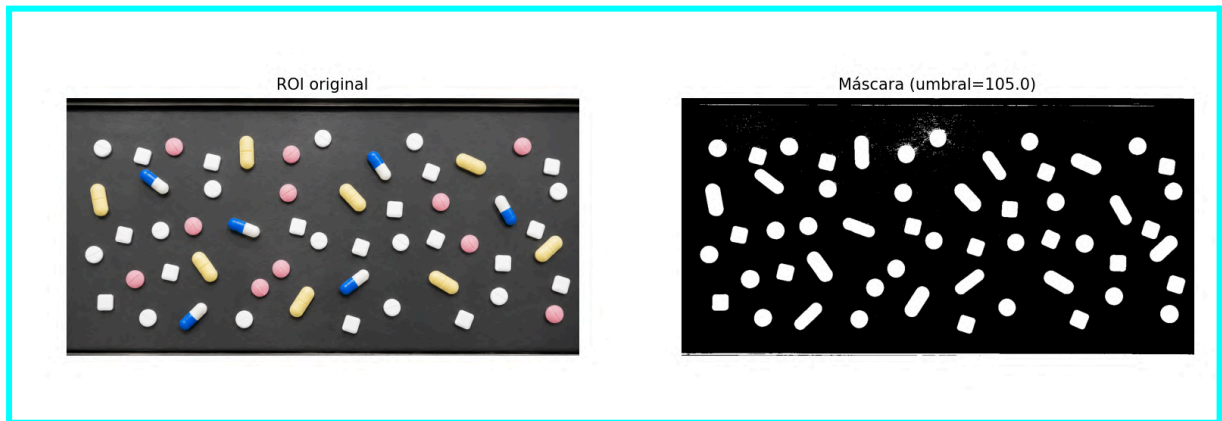


Exploramos imágenes con los diferentes canales, y vemos que obtenemos una buena diferenciación con el canal V.

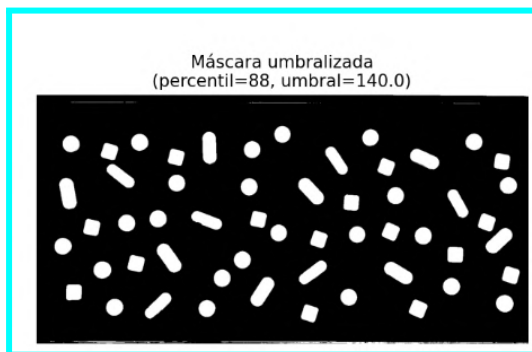
histograma del canal V



Trabajamos con percentiles y exploramos heurísticamente con qué percentil teníamos que quedarnos.. para poder obtener una buena diferenciación , suponiendo que las mezcla de pastillas y distribuciones sea similar en el futuro entendemos que es una buena solución para generalizar.



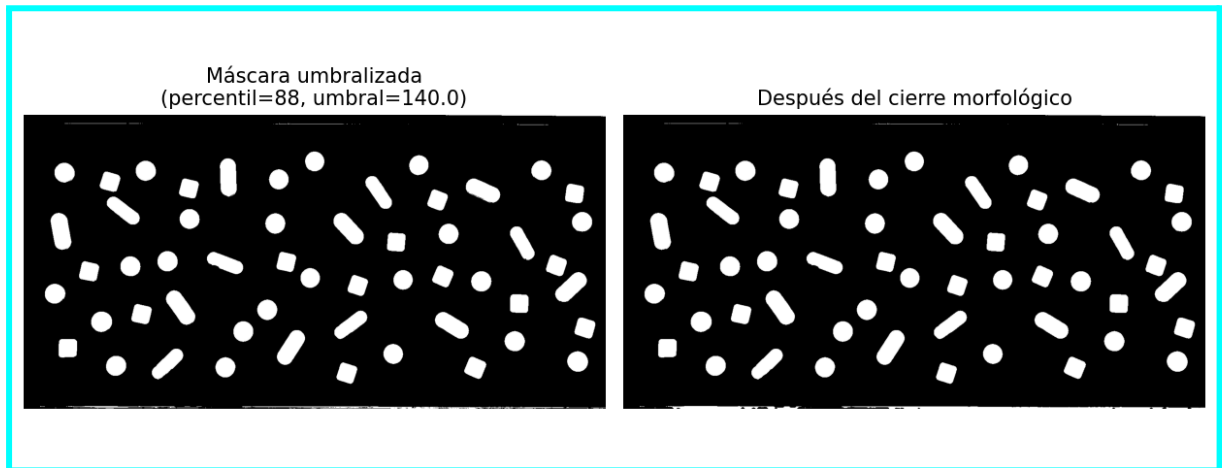
Elección del umbral.



Cierre morfológico.

A los fines de mejorar la detección de los bordes y quitar ruido que veíamos que se generaba procedemos a realizar un cierre morfológico con la idea de quitar puntos negros sobre superficies blancas.. que pudieran arrojar falsos positivos en la detección de las pastillas.

umbralización y cierre morfológico



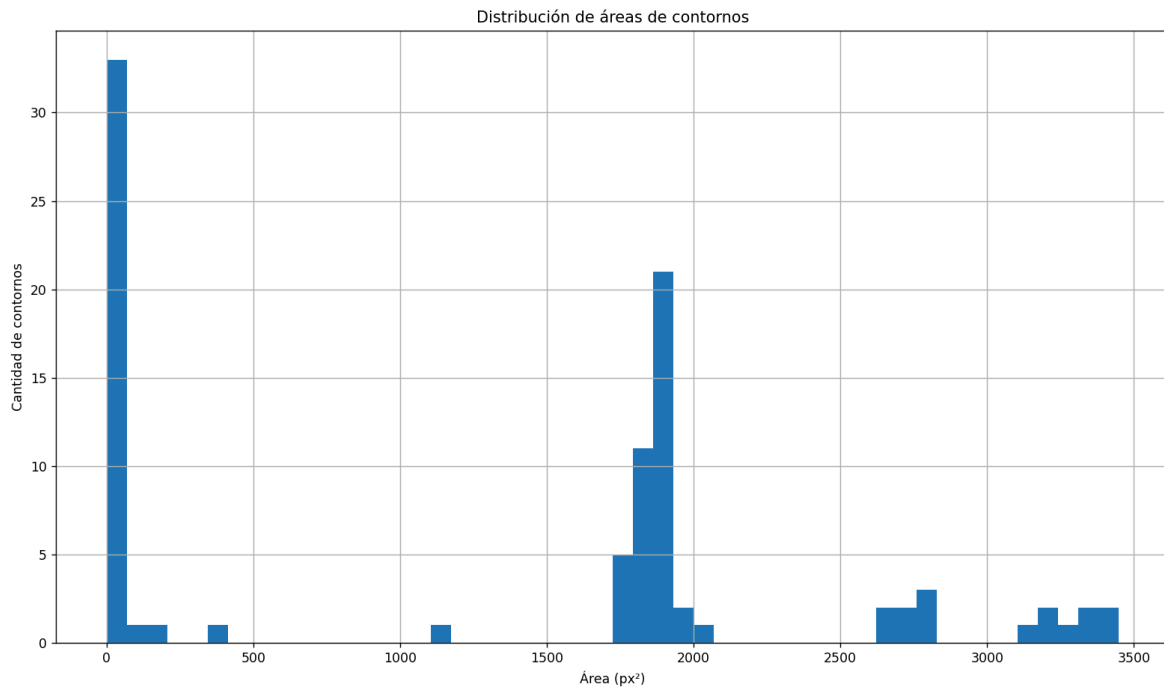
Detección de contornos.

utilizando las opciones de OpenCV ejecutamos la detección de contornos,

```
# --- Busco contornos ---  
contornos, _ = cv2.findContours(mask_clean, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

Obteniendo buenos resultados iniciales, pero también detección de falsos positivos., por lo que vamos a limitar cuales son los contornos válidos





A los fines de descartar aquellos que no nos interesen decidimos establecer una relación de los objetos con la superficie total del área de la ROI, y una serie de parámetros morfológicos adicionales para descartar objetos que no son de interés tomando el área mínima y la relación de aspecto a esos fines.

```
# --- Filtro por área mínima Y relación de aspecto ---
area_minima = 0.0014 * area_roi
contornos_filtrados = []

for c in contornos:
    area = cv2.contourArea(c)
    if area < area_minima:
        continue
    x, y, w, h = cv2.boundingRect(c)
    aspect_ratio = w / h # ancho / alto
    if aspect_ratio > 10: # si es 10 veces más ancho que alto → es el borde, no una pastilla
        continue
    contornos_filtrados.append(c)
```

Obteniendo los resultados buscados.

contornos encontrados



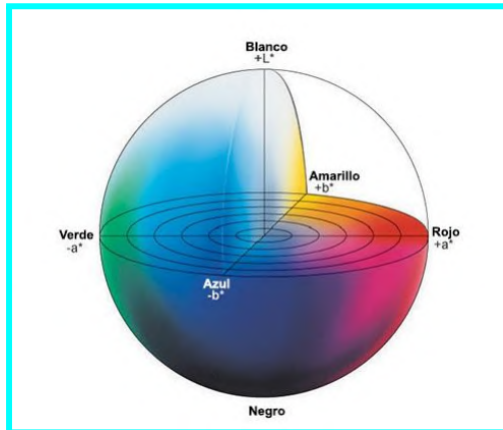
¿Cómo diferenciar las pastillas?

De la observación de las pastillas podemos decir que las mismas se diferencian por dos características, una es forma y otra es color, pero dado este universo particular entendemos que nos conviene en un primer momento diferenciar por color y luego aquellas que tienen el mismo color diferenciarla por forma.

El color importa

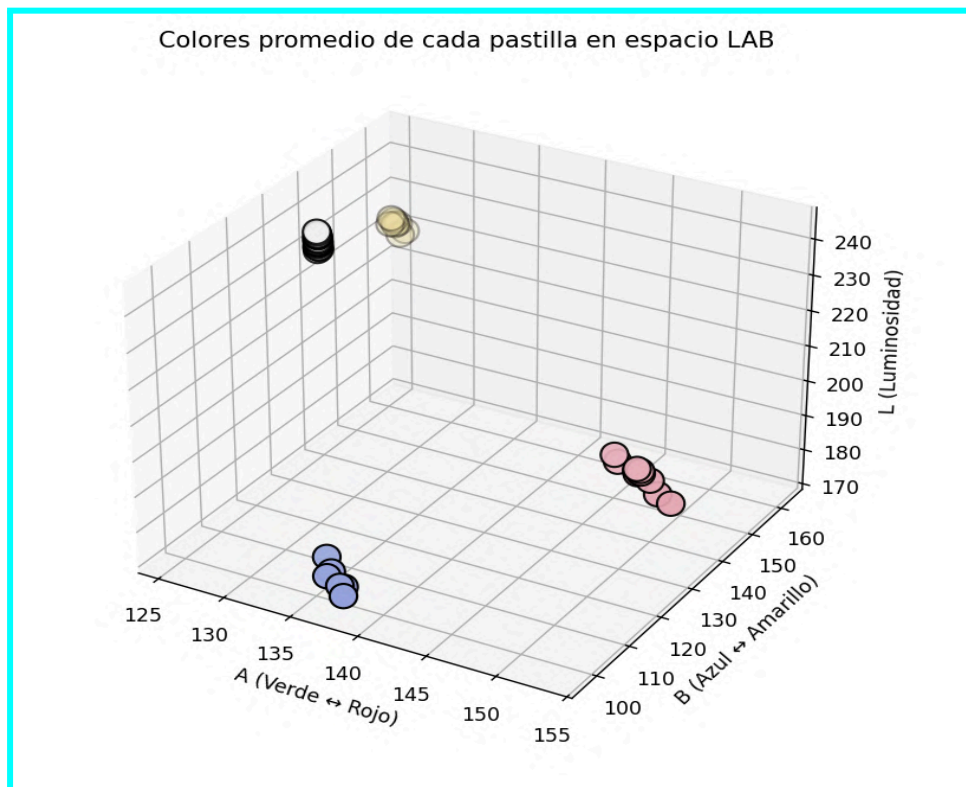
¿Cuál es el color de cada objeto para diferenciarlos? vemos que hay pastillas monocromáticas y están aquellas que tienen más de un color (en este caso 2) ahora como estamos hablando de una imagen en realidad cada objeto tiene múltiples colores, en la rosa hay muchos tonos de rosa, en la blanca distintos tonos de blanco, para poder homogeneizar el color vamos a tomar el color promedio de cada objeto, y de esa forma estandarizar a que segmentos del espacio de color pertenece cada grupo de pastillas.

espacio de color CIELAB



El resultado de analizar color promedio del objeto por el espacio de color CIELAB , vemos que encontramos 4 grupos claramente diferenciados, en esta caso los grupos se ven claramente diferenciados, en el espacio de color CIELAB, pero una opción si las condiciones de luminosidad cambiaran en distintas zonas de la imagen sería trabajar solo con los canales A y B , e independizarse del nivel de luminosidad. .

visulización de los colores promedio encontrado de los objetos en el espacio de color CIELAB



En este caso no estamos clasificando a ciegas porque tenemos a la vista como humanos, la imagen con las pastillas confirma que en lo que se refiere a color promedio, tenemos 4 tipos de pastillas en cuanto colación, esa confirmación con la realidad y la percepción humana nos parece importante a los fines de verificar y poner en marcha el sistema de detección automático.

La forma importa.

En el caso de las pastillas más claras, tenemos que resolver el problema de la forma, ¿cómo separamos las cuadradas de las redondas?

La respuesta que encontramos es segmentar las mismas en dos grupos, por eso recurrimos a trabajar con un índice de circularidad.

Aquellas con un índice de circularidad mayor a un umbral serán las redondas, y las restantes serán las no redondas.

Exploración con el índice de circularidad sobre los objetos blancos.



Etiquetado de objetos para diferenciar, cuadrados dentro de los objetos blancos



¿Cómo resolvimos la segmentación de formas y colores? ¿Qué problemas tuvimos?

En un primero momento resolvimos ambas cuestiones inspirados en la **función k-means** que permite indicar en cuantos grupos queremos segmentar una serie de valores, por algún criterio, es decir se le indica el criterio y la función encontrará la mejor combinación de agrupamiento dentro del rango de valores. Así pues en un punto le decíamos que los valores promedio de las pastillas eran 4 , y luego en el caso de las pastillas blancas por intermedio de un índice de circularidad le indicamos que queríamos segmentar en dos. Esto funcionaba muy bien, teniendo una tasa de aciertos del 100 % para esta combinación de colores y formas, pero pensamos que si la variedad de colores o la variedad de formas fuera distinta podría fracasar porque al no ser todos los valores tanto en color como en forma exáctamente iguales en la imagen podría forzar la creación de grupos en donde nos los existía.

¿Qué solución encontramos?,

Para este caso y teniendo en cuenta la naturaleza del problema decidimos crear reglas de clasificación basadas en color y forma, dejando umbrales definidos.

Así pues para el color y estando en el espacio de color CIELAB decidimos usar esta función a los fines de hacer una clasificación por el color de los objetos de interés.

Resolución de la identificación del color

```
def identificar_grupo_color(roi, contorno):  
    # 3. Extraer promedios ignorando el fondo  
    media_lab = cv2.mean(roi_lab, mask=mask_pastilla)  
    media_hsv = cv2.mean(roi_hsv, mask=mask_pastilla)  
  
    # 4. Extraer canales (OpenCV ubica el neutro en 128 para uint8)  
    canal_a = media_lab[1] # Verde < 128 < Rojo/Rosado  
    canal_b = media_lab[2] # Azul < 128 < Amarillo  
    brillo_v = media_hsv[2]  
    saturacion_s = media_hsv[1]  
    # --- LÓGICA DE UMBRALADO CROMÁTICO PÚRO ---  
    if canal_a > 140:  
        # Fuerte componente rojiza  
        return 'RR'  
  
    elif canal_b > 138:  
        # Fuerte componente amarilla  
        return 'AP'  
  
    elif canal_b < 120:  
        # Fuerte componente azulada (tira el promedio por debajo de 128)  
        return 'AzC'  
  
    else:  
        # Si A y B son neutros, es un color acromático. Validamos que sea blanca  
        # (alto brillo, baja saturación) para no confundir con sombras.  
        if brillo_v > 120 and saturacion_s < 60:  
            return 'blanca'  
        else:  
            # Color no reconocido o sombra  
            return 'indefinido'
```

y en el caso de las pastillas que en el primer paso clasificamos como blancas, decidimos separar la circularidad mediante la implementación de un índice de circularidad,

Resolución de la clasificación por forma, de los objetos ya clasificados como blancos.

```
def clasificar_blancas_por_forma(contornos_filtrados, labels, grupo_a_tipo):
    """
    Subdivide las pastillas blancas en redondas (BR) y cuadradas (BC)
    usando un umbral estático de circularidad geométrica.
    """

    # Buscamos cuál es el ID numérico asignado a las pastillas blancas
    grupo_blanco = [k for k, v in grupo_a_tipo.items() if v == 'blanca'][0]
    indices_blancas = np.where(labels.flatten() == grupo_blanco)[0]

    labels_forma = {}

    for i in indices_blancas:
        c = contornos_filtrados[i]
        area = cv2.contourArea(c)
        perimetro = cv2.arcLength(c, True)

        # # Prevención de división por cero
        if perimetro == 0:
            circularidad = 0
        else:
            circularidad = 4 * np.pi * area / (perimetro ** 2)

        # # Clasificación basada en un umbral de circularidad
        # # Círculo ideal = 1.0, Cuadrado ideal = 0.785

        if circularidad > 0.88:
            labels_forma[i] = 'BR' # Blanca Redonda
        else:
            labels_forma[i] = 'BC' # Blanca Cuadrada

    return labels_forma
```

Para los objetos con perímetro dentro del grupo de las blancas, se explora la relación mediante el índice de circularidad, sabiendo que será 1 para un perfecto, como en la realidad, y en las imágenes de realidad no podemos hablar de perfección trabajamos con holgura sobre ese valor, considerando que es un círculo que al menos llegue a un índice mayor de 0,88, los objetos analizados que no lleguen a este valor en el índice serán considerados “no redondos” quedando en el caso puntual esta categoría asignado para los objetos cuadrados.

Esta resolución tiene limitaciones, si aparecieran pastillas de otras formas y colores podría no funcionar, pero responde bien al universo de pastillas actuales, incluso si no todas estuvieran en la imagen, y dando advertencia si alguna de las mismas estuvieran fuera del rango de colores, en los colores si bien trabajamos con esta solución, en el cual se establece umbral, podría para una mayor precisión en el ajuste fino trabajar con un segmento de los canales de color, a los fines de acotar la clasificación.

¿Cuántas pastillas hay de cada una?

Llegado este punto ya estamos en condiciones de realizar nuestra clasificación y conteo de pastillas.

Para ellos definimos como llamaremos a cada tipo de pastillas , y serán denominadas

BR = las que sean blancas y redondas

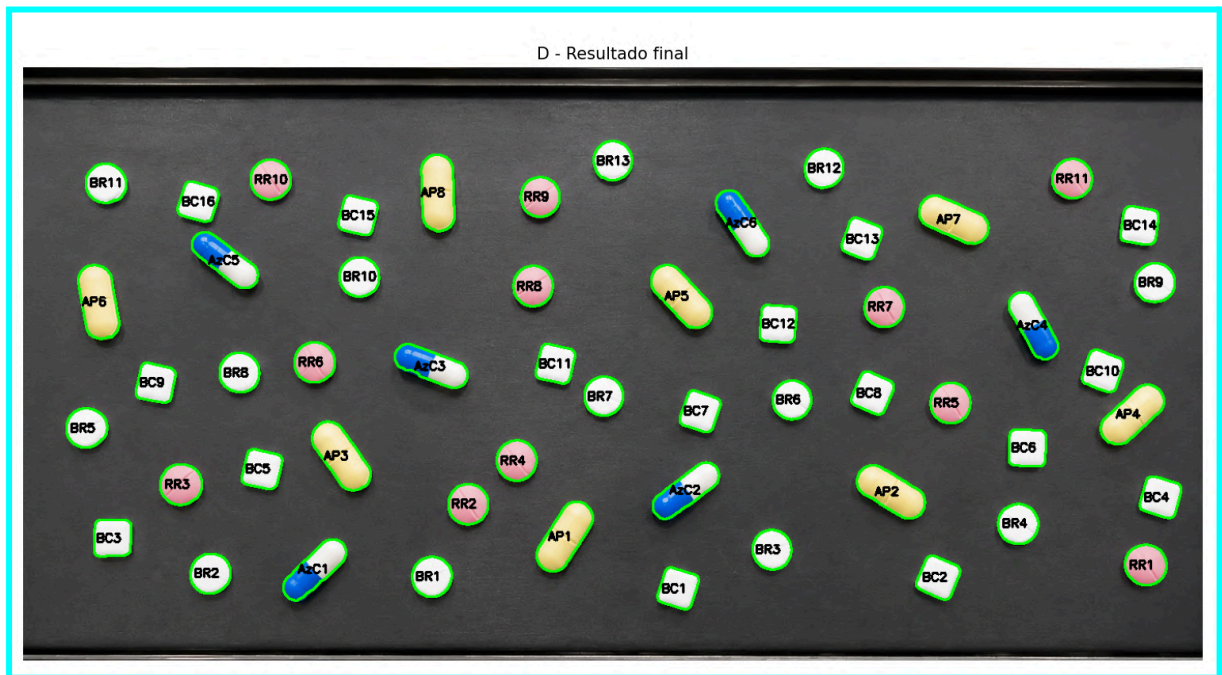
BC = las que sean blancas y cuadradas

RR = las que sean rosas y redondas

AzC = la que tenga un componente azul y sean cápsulas.

y la resolución fue hacer un bucle For en donde se analice cada objeto, se va clasificando cada uno, dentro de las categorías asignadas asignando un número de orden, obteniendo el resultado buscado tanto en forma gráfica como por consola.

visualización de la imagen final del resultando de clasificación de las pastillas.



Resultado obtenido por consola

```
=== RESULTADOS DE CLASIFICACIÓN ===  
BR: 17 pastillas  
BC: 12 pastillas  
AP: 8 pastillas  
RR: 11 pastillas  
AzC: 6 pastillas  
TOTAL: 54 pastillas
```

Reflexión final al problema 1.:

El problema nos permitió reformar las técnicas para segmentar imágenes, aplicar técnicas de mejorado de la imagen orientadas a poder detectar distintos tipos de objetos según sus características, cromáticas y morfológicas. Si bien esta es una solución para la imagen planteada se han hecho generalizaciones a los fines de que pueda ser un script válido para imágenes similares. En un caso real, donde el universo de colores y formas sea otro deberían hacerse adaptaciones para mantener los resultados obtenidos.

PRESENTACIÓN DEL PROBLEMA 2 :

Dada una serie de imágenes que incluyen en la captura fotográfica una única patente con formato argentino de nueva generación (formato mercosur), se nos solicita poder segmentar la imagen de la patente, como así la identificación de los caracteres incluidas en ella.

Conjunto de imágenes a procesar

Detección de patentes



PRESENTACIÓN DE LA SOLUCIÓN.

A los fines de explicar la solución haremos un seguimiento de un caso de éxito principal, y comentaremos en los casos de en lo que la solución propuesta no hubiera funcionado, correctamente, comentaremos las acciones correctivas que intentamos, la premisa con las que nos manejamos fue que si no podíamos tener una tasa de éxito del 100 % , consideramos que tener una tasa de éxito por encima del 80 % representa una solución aceptable para el caso .

La solución se organizó en dos etapas principales. En primer lugar, se detecta dentro de la imagen completa la región con mayor probabilidad de contener la patente. Luego, se recorta dicha región y se trabaja sobre ella para realizar la segmentación de los caracteres.

Para localizar la patente no se intenta reconocer directamente su contenido alfanumérico. En cambio, se buscan regiones rectangulares horizontales cuyas dimensiones y relación de aspecto sean compatibles con el formato de una patente Mercosur. Los candidatos obtenidos se evalúan mediante distintos criterios: cercanía con la proporción esperada, presencia simultánea de píxeles claros y oscuros, existencia de contornos internos con forma semejante a caracteres y presencia de la franja azul superior característica de este tipo de patente.

Como cada imagen constituye un caso independiente, para cada archivo se ejecuta la secuencia completa de procesamiento: carga y redimensionamiento de la imagen, preprocesamiento, detección de bordes, extracción de regiones candidatas, asignación de puntajes, selección del mejor candidato, recorte de la patente y segmentación de sus caracteres. Finalmente, se visualizan los resultados obtenidos en cada caso.

Cuando el detector principal encuentra candidatos, pero el mejor de ellos obtiene un puntaje bajo, se ejecuta de manera subsidiaria un método alternativo de detección. Este segundo procedimiento utiliza una transformación BlackHat, el gradiente Sobel en el eje X, una binarización mediante el método de Otsu y operaciones morfológicas, con el objetivo de agrupar los caracteres oscuros de la patente en una única región candidata. El resultado alternativo solo reemplaza al principal cuando consigue un puntaje superior.

IMPLEMENTACIÓN DE LA SOLUCIÓN.

El punto de partida, la imagen.

Se nos da una serie de imágenes, que en su composición presentan imágenes de patentes, el objetivo es intentar encontrar la patente y segmentar los caracteres en ella incluidas

Quitando complejidad y mejorando la imagen

Inicialmente la imagen fue convertida a escala de grises para reducir complejidad, nuestro objetivo primario es detectar bordes que sean compatibles con la relación de aspecto de un patente. (del formato actual)

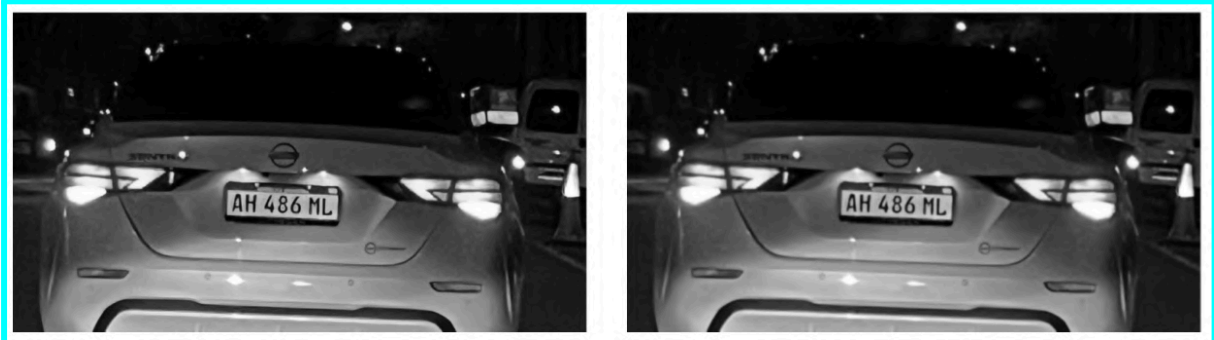
Conversión de la imagen a escala de grises (reducción de la complejidad)



Suavizado leve.

Con el objetivo de eliminar ruido pequeño, sin eliminar posibles bordes realizamos un suavizado leve sobre la imagen, para ello utilizamos un filtro gaussiano, con un kernel pequeño de 3x3.

Acción de suavizado leve



sharpening

Luego del suavizado leve buscamos remarcar aún más las zonas de transición entre claras y oscuras, por lo que recurrimos a una operación de "Sharpening" logrando un así un mayor realce de la imagen, esta operación nos da un mayor contraste pero nos genera algo de ruido

realce de la imagen realizando operación de “sharpening”



Nuevo suavizado - suavizado “ menos suave” que el anterior

Así pues una vez realizado el filtrado suave, y el realce de la transición entre zonas claras y oscuras, volvemos a realizar suavizado de la imagen, pero esta vez utilizando un kernel mayor, el objetivo es la reducción de ruido que se hubiera podido generar en la operación anterior.

suavizado de la imagen aplicando filtro Gaussiano de 5 x 5 objetivo eliminar ruido



Las tareas anteriores realizadas anteriormente de suavizado , Sharperrig, y vuelta a suavizar fueron tareas preliminares para concentrarnos en la detección de bordes.

Detección de bordes con Canny

Aquí con la imagen ya preparada, utilizamos el Canny para la detección de bordes., los parámetros de Canny en cuanto al límite inferior y superior los fuimos probando heurísticamente hasta tener valores compatibles con la tasa de éxito mínima buscada para el problema en general.



Cuales podrían ser patentes.

Sobre nuestra salida de lo obtenido en Canny utilizamos la función `findContours()`, en esta función imponemos una serie de restricciones a los fines de limitar los contornos de la imagen a aquellos que tiene determinadas características , a estos contornos candidatos lo marcamos con la caja contenedora, dentro de ellas apostamos a que se encuentra la zona de la patente

parámetro el contorno candidato debe superar para pasar a la siguiente etapa

```
# Filtro geométrico de candidatos
RATIO_MIN = 2.0
RATIO_MAX = 4.5
AREA_REL_MIN = 0.005
AREA_REL_MAX = 0.05
```

```
=====
Procesando: tp2-pruebas/img_8.jpg
=====
```

```
Shape redimensionada: (422, 800, 3) (escala=0.781)
```

```
total contornos encontrados: 112
```

```
Candidatos tras filtro ratio+área: 5
```

implementación de la limitación de candidatos conforme a los parámetros que nos permiten pasar de 112 contornos a solo 5

```
def extraer_candidatos(edges, img_area):
    for cnt in contornos:
        x, y, w, h = cv2.boundingRect(cnt)

        if h == 0:
            continue

        ratio = w / h
        area_rel = (w * h) / img_area

        if RATIO_MIN < ratio < RATIO_MAX and AREA_REL_MIN < area_rel < AREA_REL_MAX:
            candidatos.append((x, y, w, h, ratio, area_rel, cnt))

    candidatos = sorted(candidatos, key=lambda c: c[5], reverse=True)
    print(f" Candidatos principales: {len(candidatos)}")

    return candidatos
```

Contornos encontrados dadas las restricciones impuestas, dentro de ellos esperamos que esté la patente buscada



Sistema de Scoring

a los contornos que pasaron el primero proceso de selección sometimos a la imagen a una serie de pruebas, que en caso de superarlas le daremos un puntaje, este puntaje se acumula con cada prueba, y el ganador será aquel que obtenga mayor puntaje (también incluimos una regla de desempate)

Las pruebas a las que sometemos a este grupo reducido de zonas candidatas a tener la patente son las siguientes

- ¿Tiene proporción de patente?
- ¿Tiene fondo claro y elementos oscuros?
- ¿Parece contener caracteres?
- ¿Tiene una franja azul arriba?

Por cada respuesta correcta que se ajuste a los parámetros dados suma un punto y sigue con la siguiente..

proceso de scoring, en donde los contornos candidatos van acumulando puntos, para quedarnos con el que más puntos logra al final de las pruebas.

ID	ratio_usado	angulo	dist_ratio	puntos
0	3.47	-90.00	0.39	1
1	5.27	-86.02	2.19	0
2	8.85	-16.96	5.77	0
3	2.35	-87.27	0.73	0
4	2.27	-90.00	0.81	0

ID	solap_blanco	solap_negro	puntos
0	0.027	0.332	1
1	0.048	0.410	0
2	0.154	0.746	0
3	0.310	0.387	0
4	0.384	0.347	1

ID	contornos_car	puntos
0	7	2
1	3	1
2	0	0
3	0	0
4	0	1

ID	solap_azul	puntos
0	0.268	3
1	0.149	1
2	0.000	0
3	0.000	0
4	0.000	1

Puntajes finales:

ID 0: 3 puntos <-- GANADOR

ID 1: 1 puntos

ID 2: 0 puntos

ID 3: 0 puntos

ID 4: 1 puntos

parámetros utilizados para las pruebas de scoring .

```
# Scoring: ratio
RATIO_IDEAL    = 3.08
UMBRAL_ANGULO  = 15      # grados; por encima se corrige con minAreaRect
DIST_RATIO_MAX = 0.5     # tolerancia alrededor del ratio ideal

# Scoring: píxeles blancos y negros
UMBRAL_BLANCO  = 200
UMBRAL_NEGRO   = 100
UMBRAL_SOLAP_BLANCO = 0.40
UMBRAL_SOLAP_NEGRO  = 0.10

# Scoring: contornos de caracteres dentro del candidato
RATIO_CAR_MIN  = 1.2     # hc/wc mínimo (más alto que ancho)
RATIO_CAR_MAX  = 2.5
AREA_CAR_MIN   = 0.04    # fracción del roi
AREA_CAR_MAX   = 0.20
N_CAR_MIN      = 3       # mínimo de contornos con forma de carácter
N_CAR_MAX      = 7       # máximo (la patente tiene 7 caracteres)

# Scoring: franja azul superior
UMBRAL_SOLAP_AZUL = 0.25 # fracción mínima de píxeles azules en la franja superior
PROPORCION_FRANJA = 0.20 # fracción del alto del candidato que se evalúa
# Rango HSV del azul de la patente Mercosur argentina
AZUL_HSV_BAJO = np.array([100, 80, 50])
AZUL_HSV_ALTO = np.array([130, 255, 255])
```

Contorno ganador, para la imagen analizada



Un camino alternativo cuando el scoring es bajo

En algunos casos, el método principal encontraba candidatos, pero ninguno obtenía un puntaje suficientemente alto. Cuando el mejor candidato tenía menos de dos puntos, se ejecutaba un segundo método de detección.

Este método alternativo comienza aplicando una operación BlackHat sobre la imagen en escala de grises. Esta operación ayuda a resaltar elementos oscuros sobre fondos claros, como los caracteres negros de la patente.

Luego se aplica el operador Sobel en el eje horizontal. Como los caracteres producen varios cambios de intensidad de izquierda a derecha, este paso permite destacar sus bordes verticales.

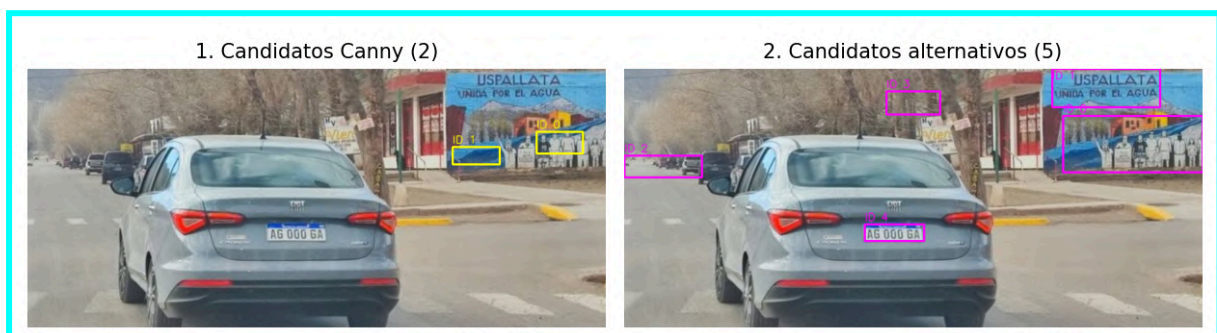
El resultado se suaviza con un filtro gaussiano y después se binariza utilizando el método de Otsu, que calcula automáticamente un umbral para separar las regiones de interés del fondo.

A continuación, se realiza un cierre morfológico con un elemento estructural rectangular. El objetivo es unir los bordes de los distintos caracteres y formar una única región horizontal. Después se aplica una dilatación para reforzar esa región y cerrar pequeños espacios que todavía puedan quedar separados.

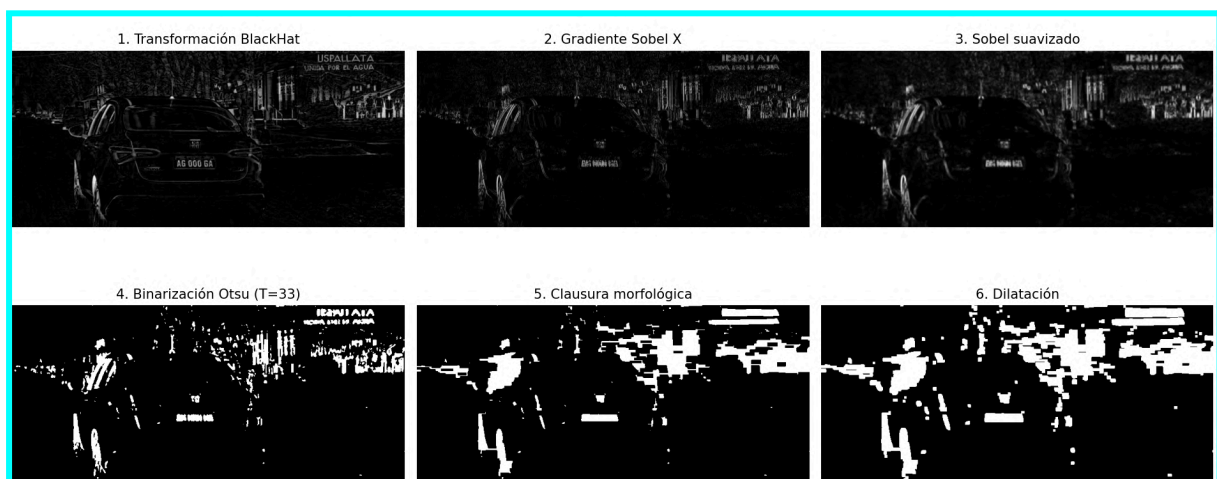
Finalmente, se buscan los contornos externos y se vuelven a filtrar según su relación de aspecto y su tamaño. Los candidatos obtenidos con este método también son evaluados con el mismo sistema de puntajes utilizado en el camino principal.

El resultado alternativo solo reemplaza al anterior cuando obtiene un puntaje mayor. De esta manera, el segundo método funciona como una opción de respaldo para los casos en los que la detección inicial no resulta suficientemente confiable conforme al puntaje mínimo que nosotros establecimos.

Caso donde el camino principal por debajo del límite de aceptación y se recurre al camino alternativo obteniendo mejores resultados

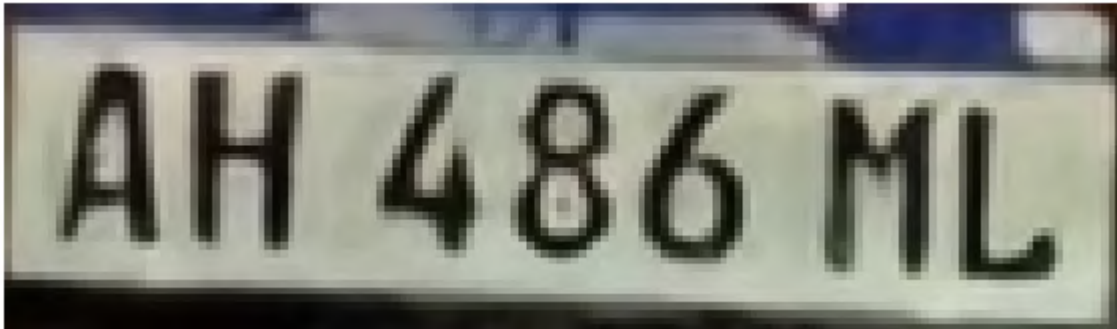


Transformaciones que realiza el camino alternativo



Recortando la patente elegida (o la que creemos que es)

Una vez seleccionado el mejor candidato, se realizó el recorte de la región correspondiente a la patente, sobre el cual procuraremos segmentar los caracteres.



Segmentando caracteres.

Trabajamos sobre el recorte de la patente en escala de grises. Para separar los caracteres del fondo se aplicó una umbralización adaptativa gaussiana invertida. De esta manera, las letras y los números oscuros pasan a verse en blanco sobre un fondo negro.

Se eligió una umbralización adaptativa porque la iluminación no es igual en toda la patente. Puede haber sombras, reflejos o zonas más claras, por lo que el valor utilizado para separar el fondo de los caracteres se calcula de manera local.

Para buscar el mejor resultado se probaron distintos tamaños de vecindad: 7, 9, 11, 13 y 15 píxeles. En cada caso se buscaron los contornos presentes en la imagen binaria y se obtuvo la caja contenedora de cada uno.

Luego descartamos los contornos que eran demasiado pequeños, demasiado grandes o que no tenían una forma similar a la de una letra o un número. Para esto se tuvo en cuenta su relación entre alto y ancho y el espacio que ocupaban dentro del recorte.

Finalmente, tomamos el resultado cuya cantidad de posibles caracteres estaba más cerca de los siete caracteres esperados en una patente Mercosur. Los caracteres seleccionados se marcaron con rectángulos sobre la imagen para poder visualizar el resultado.



Pruebas ejecutadas:

```
42
43 # Segmentación de caracteres (Parte B)
44 RATIO_SEG_MIN = 1.2
45 RATIO_SEG_MAX = 2.5
46 AREA_SEG_MIN = 0.03
47 AREA_SEG_MAX = 0.20
48
```

Con las primeras medidas de área y ratio que ejecutamos obtuvimos resultados parciales, dado que se encontraban todas las letras en 6 de las 12 imágenes, en el resto teníamos problemas para detectar los contornos, luego de sucesivas iteraciones terminamos encontrando que los mejores resultados se obtenían a partir de las siguientes medidas:

```
42
43 # Segmentación de caracteres (Parte B)
44 # RATIO_SEG_MIN = 1.2
45 # RATIO_SEG_MAX = 2.5
46 # AREA_SEG_MIN = 0.03
47 # AREA_SEG_MAX = 0.20
48
49 RATIO_SEG_MIN = 1.0
50 RATIO_SEG_MAX = 4.0
51 AREA_SEG_MIN = 0.02
52 AREA_SEG_MAX = 0.25
53
```

Ratio entre 1.2 y 2.5 – Área entre 0.03 y 0.20



Ratio entre 1.2 y 2.5 – Área entre 0.03 y 0.20



Éxitos y Fracasos

Nuestra implementación logró una tasa de éxito del 83,33 %, probamos otras combinaciones de parámetros, o iteraciones en aquellos parámetros dinámicos pero presentada fue la más exitosa. Los casos en los que no tuvimos en algunos casos pudieron ser incluidas, pero no si producirse una pérdida de otra imagen.

```
=====
RESUMEN FINAL
=====
```

Imagen	Éxito	Puntaje	Caracteres
tp2-pruebas/img_1.jpg	SI	3	7
tp2-pruebas/img_2.jpg	SI	3	7
tp2-pruebas/img_3.jpg	SI	3	7
tp2-pruebas/img_4.jpg	SI	2	7
tp2-pruebas/img_5.jpg	SI	1	0
tp2-pruebas/img_6.jpg	SI	2	7
tp2-pruebas/img_7.jpg	SI	2	0
tp2-pruebas/img_8.jpg	SI	3	7
tp2-pruebas/img_9.jpg	SI	3	7
tp2-pruebas/img_10.jpg	SI	4	7
tp2-pruebas/img_11.jpg	SI	2	7
tp2-pruebas/img_12.jpg	SI	2	7

Detectadas correctamente: 10/12

REFLEXIÓN FINAL SOBRE EL PROBLEMA 2 : La detección de las patentes nos resultó un problema desafiante, si bien no alcanzamos el éxito en todos los casos, quedamos conforme con la tasa obtenida. El problema nos lleva a pensar que en situaciones de la vida real donde la captura no está normalizada es fundamental incorporar estrategias dinámicas para la resolución de los problemas.